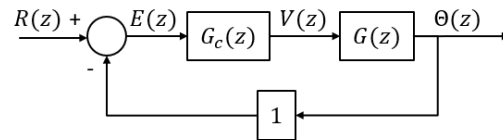




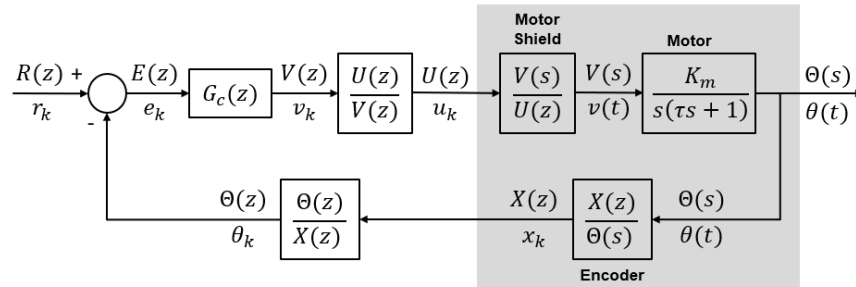
# ENG 147: Mechatronics Lab 4: Digital Position Control

Spring 2026

**Objective:** In this lab, you will implement closed-loop digital position control for your pancake motor, leveraging the motor characterization you performed in Lab 1, the voltage-to-PWM mapping you developed in Lab 2, and the digital control design strategies you explored in Lab 3.



Theoretical Control Loop (discretized plant)



Actual Implementation with Hardware

1. Plug in your battery to ensure that it is charged. Leave your charger on the 1C setting.
2. Use a piece of tape to create a “flag” on your motor shaft (as you did in Lab 0) so that you can easily observe its angular position.
3. Using MATLAB and suitable values of  $K_m$  and  $\tau$  chosen for your motor in Lab 2 or Lab 3, design a digital compensator to control shaft position subject to the following specifications:
  - Sampling time  $T_s \geq 10\text{ms}$
  - Closed-loop damping ratio  $> 0.5$
  - Zero steady state error to a reference shaft angle step input
  - | Maximum control effort  $< V_{max}$  for a  $60^\circ$  step input (**Note: check your units! You may want to work solely in radians...**), where  $V_{max}$  is the maximum motor voltage observed in Lab 1
  - Minimize 2% settling time

You may use either *design by discrete equivalent* or *direct digital design*. In your lab writeup, provide a root locus diagram and include plots demonstrating that your *digital* compensator meets the design specifications.

$$G_c(z) = \frac{V(z)}{E(z)} = \underline{\hspace{10em}} \quad T_s = \underline{\hspace{10em}}$$

4. Write the difference equation corresponding to this transfer function. All subscripts of  $v$  and  $e$  should be  $\leq k$ :

$$v_k = \underline{\hspace{10em}} \quad T_s = \underline{\hspace{10em}}$$

5. Modify your Arduino code from Lab 3 to implement closed-loop shaft position control. Your code should:
- Provide an internally-generated square wave reference input that alternates between  $+60^\circ$  and  $0^\circ$  (or similar angles), switching at a rate you specify (e.g. every 5 seconds)
  - Run a control loop consistent with the sampling time you specified above for your digital controller
  - Read from the encoder and compute shaft *position* (not velocity this time!)
  - Implement the difference equation you developed in MATLAB to compute desired control voltage
  - Convert desired control voltage to desired control PWM
  - Write desired control PWM to the motor
  - Print at least the following data columns to the serial port after some period of time (e.g. 10 seconds):
    - Timestamp
    - Control voltage
    - Control PWM
    - Shaft angle
6. Plot your experimental shaft position and control effort responses to a  $60^\circ$  (or similar) step input on the same axes as the simulated step responses. You may wish to repeat your experiment, adjusting factors including your compensator design, sampling time,  $\frac{U(z)}{V(z)}$  mapping function (think about which part of the mapping you are primarily operating in for this lab), and even your assumed motor constants  $K_m$  and  $\tau$  to identify factors that improve or degrade the correspondence between your simulated and experimental step responses. Provide plots and parameter values for your final compensator, and demonstrate whether you did or did not meet the performance specifications. If you cannot meet the performance specifications, design your compensator to get as close as possible and explain the limitation(s) you encountered.

**Writeup Guidelines and Rubric:** Your writeup should include all items and analyses specified above. Be sure to include a description and discussion of any changes you made to your compensator design or control loop in Steps 11 and 16. Don't forget to include your Arduino (C++) and MATLAB code in your PDF. Grades will be determined by assessing your combined writeup and in-person checkoff using the following rubric:

- (30 pts) Compensator design
- (40 pts) Compensator implementation and performance analysis
- (30 pts) Discussion and analysis of results

**Don't forget to embed your Arduino and MATLAB code in your lab writeup!**